UNIT I: ARM Architecture

Lecture 1: ARM Architecture Overview

Dr. Ayush Kumar Agrawal

SoCS, UPES Dehradun

ARM Architecture Overview

Lecture Objectives

After this lecture, students will be able to:

- Understand ARM architecture fundamentals
- Explain RISC principles
- Identify key features of ARM processors
- Recognize applications of ARM in embedded systems

What is ARM?

- ARM stands for **Advanced RISC Machine**
- Based on **Reduced Instruction Set Computing (RISC)**
- Designed for:
  - Low power consumption
  - High performance
  - Embedded applications
- Widely used in IoT, robotics, automation

RISC vs CISC Concept

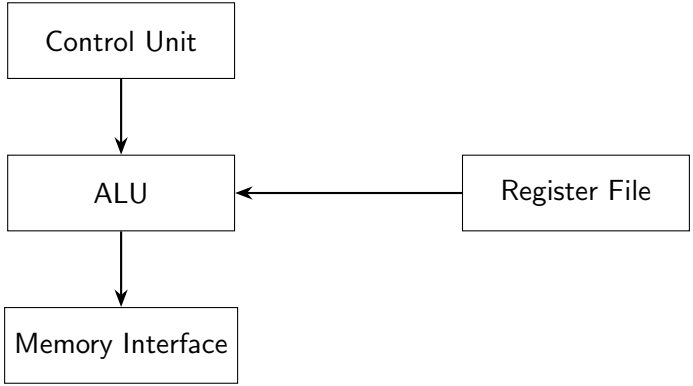
RISC

- Simple instructions
- Fixed instruction length
- Faster execution
- More registers

CISC

- Complex instructions
- Variable length
- Fewer registers
- Higher power consumption

Basic ARM Architecture Concept



- Central Processing Core
- Register-based operations
- Memory accessed via load/store instructions

Load-Store Architecture

- Data must be loaded into registers before processing
- Only Load (LDR) and Store (STR) access memory
- Arithmetic operations occur in registers

Example Concept:

```
LDR R1, [R2]

ADD R3, R1, R4

STR R3, [R2]
```

Key Features of ARM Architecture

- 32-bit architecture
- Large register set
- Pipeline execution
- Low power consumption
- High code density
- Scalable processor family

ARM Pipeline Concept

- Improves instruction throughput
- Multiple instructions overlap in execution

Applications of ARM Processors

- Embedded systems
- Industrial IoT devices
- Robotics controllers
- Mobile devices
- Automotive ECUs

Why ARM is Important for Industrial IoT

- Energy efficient operation
- Real-time performance
- Compact system design
- Scalable architecture

Lecture Summary

- ARM is based on RISC principles
- Uses load-store architecture
- Pipeline improves performance
- Ideal for embedded and IIoT systems

Quick Questions

1. What does ARM stand for?
2. What is load-store architecture?
3. How does pipeline improve performance?

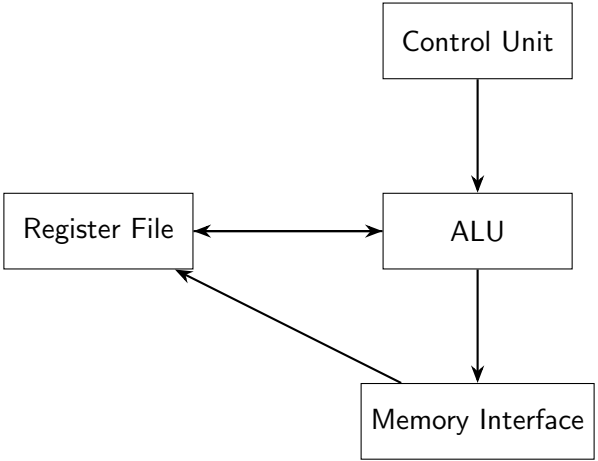
Internal Working of ARM Processor

Lecture Objectives

After this lecture, students will be able to:

- Understand internal datapath of ARM
- Explain register organization
- Interpret CPSR flags
- Describe instruction execution cycle

ARM Internal Datapath



- Register-based computation
- ALU performs arithmetic and logic

ARM Register Organization

- 16 general-purpose registers (R0–R15)
- R13 – Stack Pointer (SP)
- R14 – Link Register (LR)
- R15 – Program Counter (PC)
- CPSR – Current Program Status Register

Register Layout Diagram

R0 – General Purpose

R1 – General Purpose

...

R12 – General Purpose

R13 – Stack Pointer

R14 – Link Register

R15 – Program Counter

CPSR – Status Register

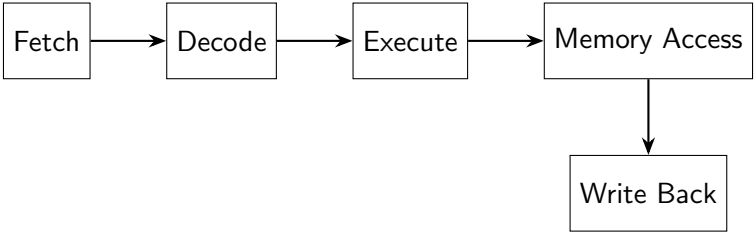
CPSR contains condition flags:

- N – Negative flag
- Z – Zero flag
- C – Carry flag
- V – Overflow flag

These flags determine conditional execution.

Example: ADD R1, R2, R3

Instruction Execution Cycle



- Each instruction passes through pipeline stages
- Improves throughput

Harvard vs Von Neumann Architecture

Harvard

- Separate instruction and data memory
- Faster access

Von Neumann

- Single memory for code and data
- Simpler design

ARM Cortex-M follows Modified Harvard Architecture.

Introduction to AMBA Bus

AMBA = Advanced Microcontroller Bus Architecture

- AHB – High-performance bus
- APB – Peripheral bus
- Enables modular system design

Lecture Summary

- ARM uses register-based architecture
- CPSR enables conditional execution
- Pipeline improves instruction throughput
- Modified Harvard architecture enhances performance

Quick Concept Questions

1. What is the role of CPSR?
2. Why is PC stored in R15?
3. Difference between Harvard and Von Neumann?

32-bit Microcontroller Benefits

Lecture Objectives

After this lecture, students will be able to:

- Explain what 32-bit architecture means
- Compare 8-bit, 16-bit and 32-bit MCUs
- Understand memory addressing capability
- Analyze performance benefits in embedded systems

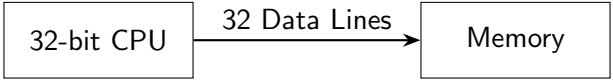
What Does 32-bit Mean?

- Data bus width = 32 bits
- ALU processes 32-bit data in one cycle
- Registers are 32-bit wide
- Address bus supports large memory space

$$2^{32} = 4,294,967,296 \text{ memory locations}$$

Maximum Addressable Memory = 4 GB

Bus Width Concept



- Wider bus transfers more data per cycle
- Improves throughput

8-bit vs 16-bit vs 32-bit Comparison

Feature	8-bit	16-bit	32-bit
Data width	8 bits	16 bits	32 bits
Max Memory	Limited	Moderate	4 GB
Speed	Low	Medium	High
Applications	Basic control	Industrial	IoT/Robotics

Arithmetic Precision

- 32-bit ALU handles large numbers directly
- Better support for:
  - Signal processing
  - Sensor fusion
  - Cryptography
- Reduces multiple-cycle operations

Example:

$$(2^{31} - 1) = 2,147,483,647$$

Memory Addressing Diagram

0x00000000
0x00000004
...
0xFFFFFFFF

- 32-bit address range
- Linear memory mapping

Performance Benefits

- Faster instruction execution
- Better multitasking capability
- Improved interrupt handling
- Efficient handling of communication stacks

Industrial and IoT Relevance

- Real-time motor control
- Edge computing devices
- Smart sensors
- Robotics controllers

32-bit MCUs enable advanced embedded intelligence.

Power vs Performance Tradeoff

- 32-bit MCUs provide:
  - Higher performance
  - Lower power (modern ARM cores)
- Ideal for Industrial IoT systems

Lecture Summary

- 32-bit refers to data width and address width
- Supports 4 GB memory addressing
- Provides higher speed and precision
- Essential for modern embedded systems

Quick Questions

1. What is the maximum memory supported by 32-bit?
2. Why is 32-bit better for DSP applications?
3. Compare 8-bit and 32-bit MCUs.

ARM vs Intel Architecture

Lecture Objectives

After this lecture, students will be able to:

- Compare ARM and Intel architectures
- Understand RISC vs CISC principles
- Analyze performance and power trade-offs
- Identify industrial use cases

RISC vs CISC Philosophy

ARM (RISC)

- Simple instructions
- Fixed instruction length
- Load-store architecture
- More general-purpose registers
- Optimized for power efficiency

Intel (CISC)

- Complex instructions
- Variable instruction length
- Memory-to-memory operations
- Fewer general registers (x86 legacy)
- High performance focus

Instruction Format Comparison

ARM Instruction (Fixed 32-bit Format)

ADD R1, R2, R3

Intel Instruction (Variable Length)

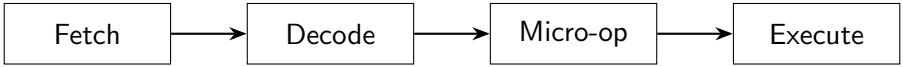
ADD AX, BX

- ARM decoding is faster
- Intel requires complex decoding logic

Pipeline Architecture Comparison

ARM

Intel



- Intel converts CISC to micro-operations internally
- ARM pipeline is simpler and power-efficient

Register Architecture Comparison

Feature	ARM	Intel
General Registers	16 (R0–R15)	Limited (x86 legacy)
Instruction Width	Fixed	Variable
Load-Store Model	Yes	No
Condition Flags	CPSR	EFLAGS

Memory Model Comparison

ARM

- Modified Harvard
- Separate I/D paths
- Faster throughput

Intel

- Von Neumann (logical)
- Unified memory model
- Complex cache hierarchy

Power Consumption Comparison

- ARM:
  - Optimized for low power
  - Used in battery-operated devices
- Intel:
  - High performance focus
  - Higher thermal output

Performance per Watt is higher in ARM-based systems.

Application Domain Comparison

ARM	Intel
IoT Devices	Desktop PCs
Robotics Controllers	Workstations
Mobile Phones	Servers
Automotive ECUs	High-end Computing

Industrial Perspective

Why ARM dominates embedded systems:

- Deterministic performance
- Lower heat dissipation
- Compact silicon design
- Scalable IP licensing model

Exam-Oriented 10-Mark Question

Compare ARM and Intel architecture in terms of:

- Instruction set
- Pipeline design
- Memory architecture
- Power consumption

Lecture Summary

- ARM follows RISC philosophy
- Intel follows CISC philosophy
- ARM offers better power efficiency
- Intel focuses on high-end computing

Bus Structure in ARM

Lecture Objectives

After this lecture, students will be able to:

- Define bus architecture in microprocessors
- Explain different types of buses
- Understand AMBA bus hierarchy
- Compare AHB, APB and AXI
- Analyze bus performance in embedded systems

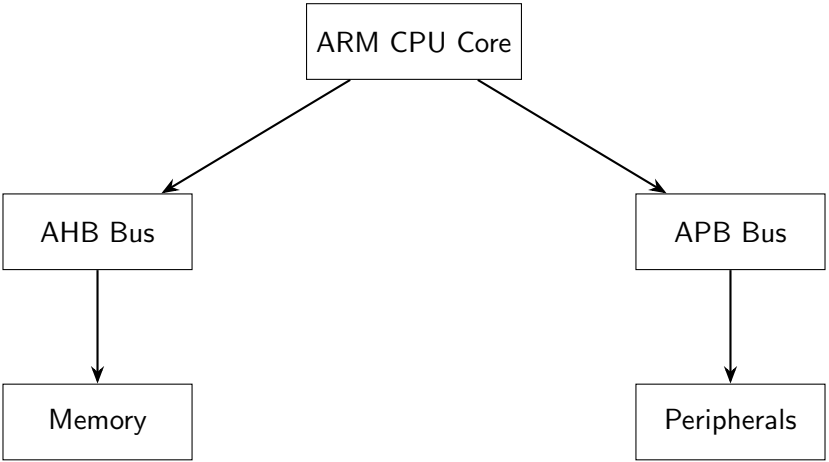
What is a Bus?

- A bus is a communication pathway used to transfer data
- Connects CPU, memory and peripherals
- Consists of:
  - Data lines
  - Address lines
  - Control lines
- Determines system performance
- Impacts latency and throughput

Types of Buses in ARM Systems

- Data Bus
  - Transfers actual data
  - Width determines performance (8, 16, 32 bits)
- Address Bus
  - Carries memory addresses
  - Determines maximum memory capacity
- Control Bus
  - Carries control signals
  - Read/Write signals
  - Interrupt signals

ARM Bus Hierarchy (Conceptual)



AMBA Architecture

AMBA = Advanced Microcontroller Bus Architecture

- Developed by ARM
- Standard on-chip communication protocol
- Modular and scalable
- Supports multi-master systems

AHB – Advanced High-performance Bus

- High-speed system bus
- Used for:
  - CPU
  - DMA
  - High-speed memory
- Supports burst transfers
- Supports pipelined operations
- Multiple bus masters supported

APB – Advanced Peripheral Bus

- Low-speed bus
- Used for:
  - UART
  - Timers
  - GPIO
  - ADC
- Simpler protocol
- Lower power consumption
- No burst transfers

AXI – Advanced eXtensible Interface

- High-performance bus
- Used in advanced ARM processors
- Supports:
  - Out-of-order transactions
  - Multiple outstanding transactions
  - Separate read and write channels
- Suitable for high bandwidth systems

Bus Arbitration

- Required when multiple masters request bus access
- Arbitration methods:
  - Fixed priority
  - Round-robin
- Ensures fair and controlled data transfer
- Prevents bus conflicts

Performance Factors

- Bus width
- Clock frequency
- Burst transfer capability
- Arbitration scheme
- Latency and throughput

System performance heavily depends on bus design.

Industrial and Embedded Relevance

- Determines real-time performance
- Affects interrupt latency
- Impacts communication stack speed
- Essential in Industrial IoT controllers

Exam-Oriented 10-Mark Question

Explain ARM bus structure with:

- Data, Address and Control bus
- AMBA architecture
- AHB vs APB comparison

Lecture Summary

- Bus connects CPU, memory and peripherals
- AMBA standardizes ARM communication
- AHB is high-speed bus
- APB is low-power peripheral bus
- Bus design affects overall performance

Register Structure in ARM

Lecture Objectives

After this lecture, students will be able to:

- Explain ARM register organization
- Differentiate general and special purpose registers
- Interpret CPSR flag bits
- Understand conditional execution

ARM Register Organization

- ARM architecture contains 16 visible registers
- Each register is 32-bit wide
- Organized as:
  - R0–R12: General Purpose Registers
  - R13: Stack Pointer (SP)
  - R14: Link Register (LR)
  - R15: Program Counter (PC)
- CPSR: Current Program Status Register

Register Layout Diagram

R0 – General Purpose

R1 – General Purpose

R2 – General Purpose

...

R12 – General Purpose

R13 – Stack Pointer (SP)

R14 – Link Register (LR)

R15 – Program Counter (PC)

General Purpose Registers (R0–R12)

- Used for arithmetic and logical operations
- Temporary data storage
- Used in function parameter passing
- Can participate in load/store instructions
- No fixed predefined role

Special Purpose Registers

R13 – Stack Pointer (SP)

- Points to top of stack
- Used in function calls and interrupts

R14 – Link Register (LR)

- Stores return address during function calls

R15 – Program Counter (PC)

- Holds address of next instruction
- Automatically increments after execution

CPSR – Current Program Status Register

- Stores condition flags
- Controls processor mode
- Indicates interrupt status

CPSR Bit-Level Structure

N	Z	C	V		Mode Bits

- N – Negative
- Z – Zero
- C – Carry
- V – Overflow

Conditional Execution in ARM

- ARM supports conditional instruction execution
- Based on CPSR flags
- Example:

```
ADDNE R1, R2, R3
```

- Executes only if Zero flag $= 0$

SPSR – Saved Program Status Register

- Used in exception modes
- Stores previous CPSR value
- Restores processor state after interrupt

Banked Registers (Concept Overview)

- Some registers are banked in different modes
- Used during:
  - Interrupts
  - Fast Interrupt Requests (FIQ)
- Improves interrupt response time

Exam-Oriented 10-Mark Question

Explain ARM register organization including:

- General purpose registers
- Special purpose registers
- CPSR structure
- Conditional execution

Lecture Summary

- ARM has 16 visible 32-bit registers
- R13, R14 and R15 are special registers
- CPSR stores condition flags
- ARM supports conditional execution

Pins and Ports in ARM

Lecture Objectives

After this lecture, students will be able to:

- Differentiate between pins and ports
- Understand GPIO architecture
- Explain pin multiplexing
- Describe port configuration registers

What are Pins?

- Physical terminals of the microcontroller
- Provide interface to external hardware
- Used for:
  - Input signals (sensors)
  - Output signals (actuators)
  - Communication protocols
- Each pin connected to internal circuitry

What are Ports?

- Logical grouping of multiple pins
- Identified as Port A, Port B, Port C, etc.
- Each port controlled by dedicated registers
- Pins inside a port can be configured independently

Example: Port A $\rightarrow$ PA0, PA1, PA2 ... PA15

GPIO Block Diagram

- GPIO controller connects internal bus to physical pins
- Controlled via memory-mapped registers

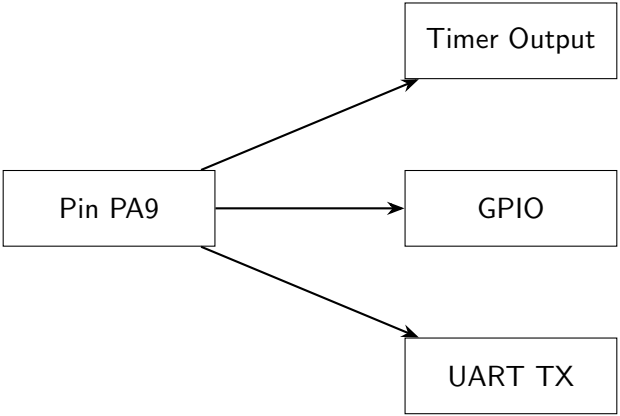
GPIO Modes

Each pin can operate in:

- Input Mode
- Output Mode
- Alternate Function Mode
- Analog Mode

Configuration done via control registers.

Pin Multiplexing Concept



- One physical pin can serve multiple functions
- Selected using Alternate Function Registers

Port Configuration Registers

Common registers (vendor specific naming may vary):

- MODER – Mode selection
- OTYPER – Output type
- OSPEEDR – Output speed
- PUPDR – Pull-up/Pull-down
- IDR – Input data register
- ODR – Output data register

All are memory-mapped registers.

Electrical Characteristics

- CMOS logic levels
- Maximum current per pin is limited
- Push-pull vs Open-drain configuration
- Internal pull-up and pull-down resistors

Improper configuration may damage the MCU.

Interrupt Capability of Pins

- GPIO pins can generate interrupts
- Edge triggered:
  - Rising edge
  - Falling edge
- Level triggered
- Used in:
  - Button press detection
  - Sensor event handling

Industrial and Embedded Applications

- Sensor interfacing
- Motor control signals
- Communication interfaces
- Human-machine interface

GPIO flexibility is critical in Industrial IoT systems.

Exam-Oriented 10-Mark Question

Explain Pins and Ports in ARM architecture including:

- GPIO modes
- Pin multiplexing
- Port configuration registers

Lecture Summary

- Pins are physical interfaces
- Ports are logical groupings of pins
- GPIO supports multiple modes
- Pin multiplexing increases flexibility

ARM Based Processor Family

Lecture Objectives

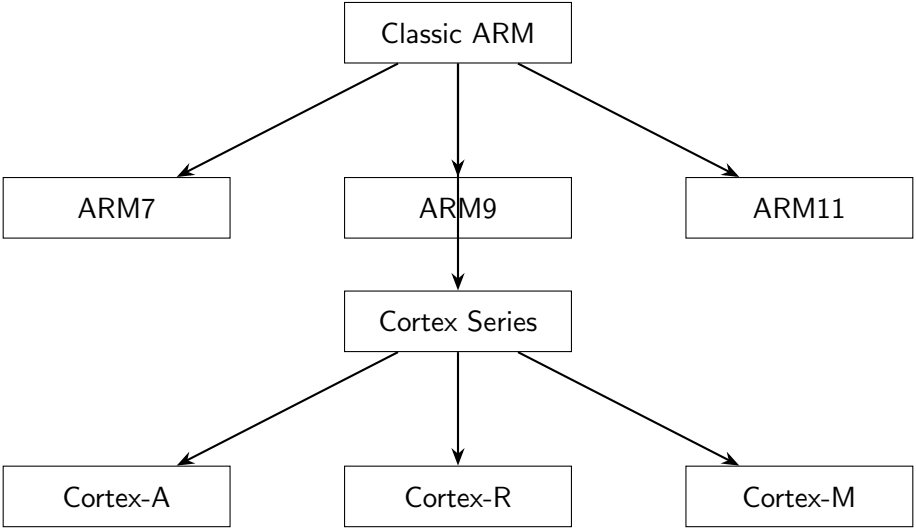
After this lecture, students will be able to:

- Understand evolution of ARM processors
- Differentiate ARM7, ARM9, ARM11
- Explain Cortex-A, R and M families
- Identify appropriate ARM family for applications

Evolution of ARM Architecture

- ARM architecture introduced in 1980s
- Designed for low power embedded systems
- Progressed through multiple generations
- Improved:
  - Pipeline depth
  - Clock speed
  - Bus architecture
  - Interrupt handling

ARM Processor Family Tree



ARM7 Family

- Based on ARMv4 architecture
- 3-stage pipeline
- Low power consumption
- Widely used in early embedded systems
- Example applications:
  - Industrial controllers
  - Consumer electronics

ARM9 Family

- Based on ARMv5 architecture
- 5-stage pipeline
- Improved performance over ARM7
- Separate instruction and data cache
- Used in:
  - Embedded Linux systems
  - Multimedia devices

ARM11 Family

- Based on ARMv6 architecture
- Higher clock frequency
- Improved memory management
- Used in early smartphones
- Transitional generation before Cortex series

Introduction to Cortex Series

- Launched to standardize ARM architecture
- Improved performance and scalability
- Divided into:
  - Cortex-A (Application processors)
  - Cortex-R (Real-time processors)
  - Cortex-M (Microcontrollers)

Cortex-A Series

- Designed for high-performance applications
- Supports operating systems (Linux, Android)
- Features:
  - MMU
  - Cache hierarchy
  - High clock frequency
- Used in smartphones and tablets

Cortex-R Series

- Designed for real-time applications
- Deterministic execution
- Low interrupt latency
- Used in:
  - Automotive systems
  - Hard disk controllers
  - Industrial automation

Cortex-M Series

- Designed for microcontroller applications
- Optimized for low power and cost
- Integrated NVIC for fast interrupt handling
- Used in:
  - IoT devices
  - Robotics
  - Embedded control systems

Comparison Table

Feature	Cortex-A	Cortex-R	Cortex-M
Performance	High	Medium	Low-Medium
Real-time	No	Yes	Yes
OS Support	Yes	Limited	No RTOS optional
Application	Smartphones	Automotive	IoT/Embedded

Industrial Relevance

- Cortex-M dominates embedded and IoT systems
- Cortex-R used in safety-critical systems
- Cortex-A used in smart gateways
- Selection depends on:
  - Performance requirement
  - Real-time constraints
  - Power consumption

Exam-Oriented 10-Mark Question

Explain ARM processor family evolution and compare:

- ARM7, ARM9 and ARM11
- Cortex-A, R and M series

Lecture Summary

- ARM evolved from classic ARM7 to Cortex series
- Cortex-A for high performance
- Cortex-R for real-time systems
- Cortex-M for embedded applications

ARM Cortex-M Series

Lecture Objectives

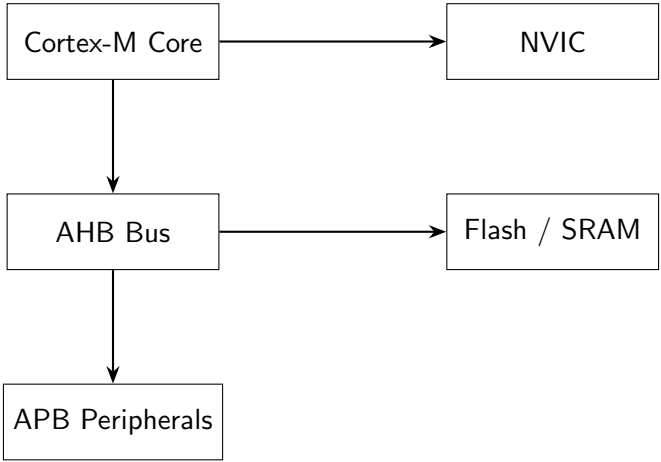
After this lecture, students will be able to:

- Understand Cortex-M architecture
- Compare M0, M3, M4, M7 cores
- Explain NVIC concept
- Describe memory organization

Why Cortex-M Series?

- Designed for microcontroller applications
- Optimized for:
  - Low power consumption
  - Low cost
  - Deterministic real-time response
- Replaced legacy ARM7 in embedded domain
- Standardized interrupt handling

Cortex-M Architecture Overview



Cortex-M Core Variants

Core	Pipeline	DSP	FPU
M0/M0+	2-Stage	No	No
M3	3-Stage	No	No
M4	3-Stage	Yes	Yes
M7	6-Stage	Yes	Yes

Thumb-2 Instruction Set

- Combination of 16-bit and 32-bit instructions
- Improves code density
- Maintains high performance
- All Cortex-M cores use Thumb mode only

NVIC – Nested Vectored Interrupt Controller

- Integrated inside Cortex-M core
- Provides:
  - Fast interrupt response
  - Nested interrupt handling
  - Priority-based execution
- Reduces latency
- Essential for real-time systems

Memory Map of Cortex-M

0x00000000 – Flash Memory

0x20000000 – SRAM

0x40000000 – Peripherals

0xE0000000 – System Control Space

Real-Time Capability

- Deterministic interrupt latency
- Hardware stacking during interrupts
- Tail-chaining feature
- Low jitter performance

Cortex-M is optimized for embedded real-time systems.

Industrial Applications

- Industrial automation controllers
- Motor control systems
- IoT edge devices
- Robotics control units

Cortex-M dominates microcontroller market.

Exam-Oriented 10-Mark Question

Explain ARM Cortex-M architecture including:

- Core variants
- NVIC
- Memory map
- Real-time features

Lecture Summary

- Cortex-M designed for microcontrollers
- Integrated NVIC ensures fast interrupts
- Thumb-2 improves code efficiency
- Widely used in embedded and IoT systems

NVIC and Interrupt Handling

Lecture Objectives

After this lecture, students will be able to:

- Define interrupt and exception
- Explain NVIC architecture
- Describe interrupt execution flow
- Understand priority and preemption

What is an Interrupt?

- An interrupt is a signal that temporarily halts normal program execution
- Triggered by hardware or software events
- CPU responds by executing an Interrupt Service Routine (ISR)
- After ISR, CPU resumes previous execution

Interrupts enable real-time responsiveness.

Types of Exceptions in Cortex-M

- Reset
- NMI (Non Maskable Interrupt)
- Hard Fault
- System Exceptions
- External Interrupts (IRQ)

All are managed through NVIC.

NVIC – Nested Vectored Interrupt Controller

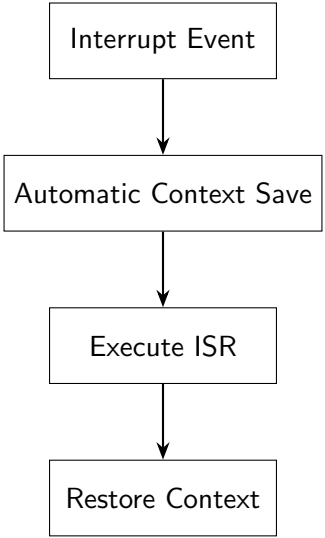
- Integrated inside Cortex-M core
- Supports multiple interrupt sources
- Provides:
  - Priority-based handling
  - Nested interrupt support
  - Low latency response
- Hardware-based interrupt management

NVIC Architecture Overview

Interrupt Priority Mechanism

- Each interrupt assigned priority level
- Lower numerical value $\rightarrow$ Higher priority
- Supports:
  - Preemption priority
  - Subpriority
- Higher priority interrupt can interrupt lower one

Interrupt Execution Flow



Hardware Stacking Mechanism

During interrupt, Cortex-M automatically saves:

- R0, R1, R2, R3
- R12
- LR
- PC
- xPSR

Stored on stack before ISR execution.
Reduces software overhead.

Tail-Chaining

- Occurs when multiple interrupts are pending
- Processor switches directly between ISRs
- Avoids unnecessary stack restore and save
- Improves performance

Reduces interrupt latency significantly.

Preemption Concept

- Higher priority interrupt can preempt lower priority ISR
- Enables deterministic real-time control
- Essential in:
  - Motor control
  - Safety systems

Interrupt Latency

- Time between interrupt request and ISR execution
- Cortex-M latency is very low
- Determined by:
  - Pipeline state
  - Priority
  - Memory speed

Low latency is critical in embedded systems.

Industrial Relevance

- Real-time motor control
- Industrial sensor monitoring
- Event-driven IoT systems
- Safety-critical automotive systems

NVIC ensures deterministic behavior.

Exam-Oriented 10-Mark Question

Explain NVIC and interrupt handling in Cortex-M including:

- Interrupt priority
- Hardware stacking
- Tail-chaining
- Preemption

Lecture Summary

- NVIC manages all interrupts
- Supports priority and nesting
- Automatic context saving
- Optimized for real-time systems

Vector Tables in Cortex-M

Lecture Objectives

After this lecture, students will be able to:

- Define vector table
- Explain its structure
- Identify its memory location
- Understand interrupt vector mapping

What is a Vector Table?

- A vector table is a memory structure
- Stores addresses of interrupt handlers
- Used by processor to jump to ISR
- Enables fast interrupt response

Each entry = Address of handler function

Why Vector Table is Required?

- Processor must know where ISR is located
- Avoids searching for handler
- Ensures deterministic response
- Essential for real-time systems

Without vector table $\rightarrow$ No interrupt handling possible

Default Location of Vector Table

- Located at address:

$$0x00000000$$

- First region of Flash memory
- Can be relocated using VTOR register

Structure of Vector Table

0x00000000 – Initial Stack Pointer

0x00000004 – Reset Handler

0x00000008 – NMI Handler

0x0000000C – Hard Fault

... System Exceptions ...

... External Interrupts ...

First Two Entries (Very Important)

Entry 0: Initial Stack Pointer

- Loaded into SP during reset

Entry 1: Reset Handler

- First instruction executed after reset
- Initializes system

These two are mandatory.

Exception Vector Order

- Reset
- NMI
- Hard Fault
- Memory Management Fault
- Bus Fault
- Usage Fault
- System service calls
- External IRQs

Order is fixed by ARM architecture.

External Interrupt Vectors

- After system exceptions
- Each peripheral interrupt has dedicated entry
- Example:
  - UART interrupt
  - Timer interrupt
  - GPIO interrupt
- NVIC uses index to access correct vector

Vector Table Relocation

- Controlled using VTOR register
- Allows vector table in:
  - SRAM
  - Bootloader region
- Used in:
  - Bootloader applications
  - Firmware update systems

Interrupt Handling Using Vector Table

- Interrupt occurs
- NVIC identifies interrupt number
- Processor calculates vector address:

$$\text{Vector Address} = \text{Base Address} + (4 \times \text{Interrupt Number})$$

- Jumps to ISR automatically

Industrial Importance

- Enables deterministic execution
- Critical in safety systems
- Used in:
  - Automotive ECUs
  - Robotics controllers
  - Industrial automation

Exam-Oriented 10-Mark Question

Explain the structure and working of Vector Table in Cortex-M including:

- Memory layout
- First two entries
- Interrupt vector calculation

Lecture Summary

- Vector table stores ISR addresses
- Located at 0x00000000
- First entry is Stack Pointer
- Used by NVIC for fast interrupt handling